10 Tipos y variables

Tipos de datos

Los tipos de datos nos **permiten categorizar y manejar diferentes valores** que podemos asignar a variables en nuestros programas de Python.

Cada tipo de dato tiene **características** y **comportamientos** específicos que nos ayudan a realizar operaciones y manipulaciones de datos de manera efectiva.

Tipos de datos básicos

- Booleanos
- Enteros
- Coma flotante
- Textos

Tipos de datos compuestos

- Listas
- Tuplas
- Diccionarios
- Conjuntos

Tipos de datos básicos:

Booleanos (bool)

Los booleanos (**bool**) son valores de verdad que representan la lógica booleana. Pueden ser True (verdadero) o False (falso) y se utilizan en expresiones lógicas y de control de flujo. Por ejemplo, en una expresión if, se evalúa si una condición es verdadera o falsa.

```
verdadero = True
falso = False
```

Números enteros (int)

Los números enteros (**int**) son números sin punto decimal, tanto positivos como negativos. Pueden ser de cualquier longitud y se

utilizan para representar cantidades enteras. Algunos ejemplos de números enteros son 5, -10, 1000, etc.

```
numero_entero = 10
```

Números de punto flotante (float)

Los números de punto flotante (**float**) son números con punto decimal. Pueden representar tanto números enteros como fraccionarios. Algunos ejemplos de números de punto flotante son 3.14, -0.5, 2.71828, etc.

```
numero_flotante = 3.14
```

Cadenas de texto (str)

Las cadenas de texto en Python (str) son secuencias de caracteres encerradas entre comillas simples (') o dobles ("). Pueden contener letras, números, símbolos y espacios. Algunos ejemplos de cadenas de texto son "Hola", 'Python', "123", etc.

```
cadena_texto = "Hola, mundo!"
```

10 Tipos y variables

Tipos de datos compuestos:

Listas (list)

Las listas en Python (**list**) son colecciones ordenadas y modificables de elementos. Pueden contener cualquier tipo de datos, incluyendo números, cadenas, listas anidadas, etc. Los elementos de una lista están indexados y se pueden acceder y modificar utilizando índices.

```
mi_lista = [1, 2, 3, 4, 5]
```

Tuplas (tuple)

Las tuplas en Python (**tuple**) son colecciones ordenadas e inmutables de elementos. Al igual que las listas, pueden contener cualquier tipo de datos. La diferencia principal es que los elementos de una tupla no pueden ser modificados una vez que la tupla ha sido creada.

```
mi_tupla = (10, 20, 30, 40, 50)
```

Diccionarios (dict)

Los diccionarios en Python (**dict**) son colecciones no ordenadas de pares clave-valor. Cada elemento en un diccionario tiene una clave única que se utiliza para acceder al valor correspondiente. Son muy útiles para representar datos estructurados y relacionados.

```
mi_diccionario = {"nombre": "Luis", "edad": 30, "ciudad":
"Madrid"}
```

Conjuntos (set)

10 Tipos y variables

Los conjuntos en Python (**set**) son colecciones no ordenadas y sin elementos duplicados. Se utilizan para operaciones matemáticas de conjunto como unión, intersección, diferencia, etc.

```
mi_conjunto = {1, 2, 3, 4, 5}
```

Variables

Las variables son contenedores que nos permiten **almacenar y manipular valores** en nuestros programas. Este valor puede ser de cualquier tipo de dato en Python (como números, cadenas de texto, listas, diccionarios, entre otros).

Internamente, el nombre de la variables actúa como un **"alias" para referirnos a una dirección de la memoria del ordenador**, en la que vamos a guardar un valor.

Declaración de variables

Al asignar un valor a una variable, Python **reserva un espacio en la memoria para almacenar este valor**.

En Python no es necesario declarar explícitamente el tipo de una variable antes de asignarle un valor (el tipo se infiere automáticamente basado en el valor que asignamos).

Por tanto, para crear una variable en Python simplemente asignamos un valor utilizando el operador =.

```
# Asignación de valores a variables
numero = 10
nombre = "Luis"
lista = [1, 2, 3]
```

10 Tipos y variables

```
# Imprimiendo los valores de las variables
print("Número:", numero)  # Salida: Número: 10
print("Nombre:", nombre)  # Salida: Nombre: Luis
print("Lista:", lista)    # Salida: Lista: [1, 2, 3]
```

En este ejemplo, hemos creado tres variables:

- **numero** que contiene el valor **10**
- **nombre** que contiene la cadena **"Luis"**
- **lista** que contiene una lista **[1, 2, 3]**.

Convenciones de nombres

Reglas para nombrar variables en Python

Durante el uso de Python, vas a tener que nombrar variables y funciones con frecuencia. No obstante, **existen ciertas reglas y buenas prácticas para seguir.**

Algunas de ellas son obligatorias (impuestas por el lenguaje), y otras son simplemente convenciones que puedes seguir.

Vamos a ver las reglas y buenas prácticas para nombrar variables en Python.

Reglas Específicas

- **Case Sentitive**: Los nombres de variables en Python son sensibles a mayúsculas y minúsculas. Por ejemplo, **variable**, **Variable** y **VARIABLE** son consideradas diferentes.

- **Inicio de Nombre**: El nombre de la variable debe comenzar con una letra (a-z, A-Z) o un guión bajo **(_)**. No debe comenzar con números ni otros caracteres.

10 Tipos y variables

```
variable = 42
_variable_privada = "Python"
1variable = "Error"  # Incorrecto, no debe comenzar con un número
```

- **Caracteres Permitidos**: Los nombres de variables pueden contener letras (a-z, A-Z), dígitos (0-9) y guiones bajos (**_**). No se permiten otros caracteres especiales como espacios, signos de puntuación o símbolos.

```
mi_variable = "Python"
edad3 = 30
mi_variable@ = "Error"  # Incorrecto, no se permite el símbolo @
```

- **Palabras Reservadas**: No se pueden utilizar palabras reservadas de Python como nombres de variables. Por ejemplo, **if**, **else**, **for**, **while**, entre otras.

```
if = 10  # Incorrecto, if es una palabra reservada
mi_if = 10  # Correcto, añadiendo prefijo o sufijo a la palabra
reservada
```

- **Legibilidad y Significado:** Es importante elegir nombres de variables que sean descriptivos y reflejen su uso en el contexto del programa. Esto mejora la legibilidad y comprensión del código.

```
total_compras = 100
num = 100  # Menos descriptivo, se desconoce el propósito
```

Convenciones de Estilo

10 Tipos y variables

Python PEP 8 (Guía de Estilo para el Código Python) sugiere convenciones adicionales para mejorar la legibilidad del código:

- **snake_case**: Para nombres de variables, funciones y métodos. Por ejemplo, **mi_variable**, **calcular_total()**.

- **CamelCase**: Para nombres de clases en Python. Por ejemplo, **MiClase**, **MiClaseEspecial**.

```
# Ejemplos de snake_case
nombre_completo = "Luis Pérez"
calcular_total = lambda x, y: x + y

# Ejemplos de CamelCase (para nombres de clases)
class MiClase:
    def __init__(self):
        self.atributo = None
```

- **Mayúsculas** para el caso de constantes, separadas por guiones bajos, siguiendo el estilo **CONSTANTE_DE_EJEMPLO**.

Conversión de tipos

En Python, es posible **convertir entre tipos de datos** utilizando funciones integradas. Algunas de las funciones de conversión más comunes incluyen:

Función	**Descripción**
int()	Convierte un valor a entero
float()	Convierte un valor a flotante
str()	Convierte un valor a cadena de texto
bool()	Convierte un valor a booleano

10 Tipos y variables

Es importante tener en cuenta que no todas las conversiones son posibles. En caso de hacer una conversión no válida, se pueden producir pérdida de información (o incluso generar un error).

Por ejemplo,

- Al convertir un flotante a entero, la parte decimal se truncará
- Al convertir una cadena de texto a un número, el valor del texto debe ser un número válido (de lo contrario, se generará un error)

Convertir un valor a entero

La función **int()** se utiliza para convertir un valor a un entero.

```
# Conversión de flotante a entero
flotante = 3.14
entero = int(flotante)

# Conversión de cadena a entero
cadena = "25"
entero = int(cadena)

# Conversión de booleano a entero
booleano = True
entero = int(booleano)
```

Si el valor no se puede convertir a un entero válido, se generará un error.

Convertir un valor a flotante

La función **float()** se utiliza para convertir un valor a un número de punto flotante

```
# Conversión de entero a flotante
```

10 Tipos y variables

```
entero = 10
flotante = float(entero)

# Conversión de cadena a flotante
cadena = "3.14"
flotante = float(cadena)

# Conversión de booleano a flotante
booleano = True
flotante = float(booleano)
```

Si el valor no se puede convertir a un flotante válido, se generará un error.

Convertir un valor a cadena de texto

La función **str()** se utiliza para convertir un valor a una cadena de texto. Esta función puede convertir valores de cualquier tipo a una representación de cadena.

```
# Conversión de entero a cadena
entero = 10
cadena = str(entero)

# Conversión de flotante a cadena
flotante = 3.14
cadena = str(flotante)

# Conversión de booleano a cadena
booleano = True
cadena = str(booleano)
```

Convertir un valor a booleano

La función **bool()** se utiliza para convertir un valor a un valor booleano. En Python, cualquier valor se puede convertir a un booleano, y el resultado puede ser True o False.

```python
# Conversión de entero a booleano
entero = 0
booleano = bool(entero)

# Conversión de cadena a booleano
cadena = "True"
booleano = bool(cadena)

# Conversión de flotante a booleano
flotante = 0.0
booleano = bool(flotante)
```

Ámbito de las variables

El ámbito o scope de una variable **se refiere a la parte del programa donde esa variable es accesible**. En Python, hay dos tipos principales de alcance de variables: local y global.

- **Variables locales**: Son aquellas definidas **dentro de una función**. Su alcance está limitado a esa función y no son accesibles fuera de ella.
- **Variables globales**: Son aquellas definidas fuera de cualquier función o en un ámbito global. Pueden ser accesibles desde **cualquier parte del programa** (tanto dentro como fuera de las funciones).

10 Tipos y variables

Variables locales

Las variables **definidas dentro de una función tienen un alcance local**. Esto significa que solo son accesibles dentro de esa función. Si intentamos acceder a una variable local desde fuera de la función, Python generará un error.

```
def mi_funcion():
    variable_local = "Hola Mundo"
    print(variable_local)

mi_funcion()
# print(variable_local)  # Esto generaría un error
```

En este ejemplo, **variable_local** está definida dentro de la función mi_funcion y solo es accesible dentro de esa función.

10 Tipos y variables

Variables globales

Las variables definidas fuera de cualquier función tienen un alcance global. Esto significa que son accesibles desde cualquier parte del programa (incluyendo dentro de funciones).

```
variable_global = "Soy global"

def imprimir_global():
    print(variable_global)

imprimir_global()  # Resultado: Soy global
print(variable_global)  # También es accesible fuera de la función
```

En este caso, **variable_global** está definida fuera de la función **imprimir_global**, por lo que es una variable global y puede ser accesible desde cualquier parte del programa.

Uso de la palabra clave global

En algunos casos, es posible que necesitemos modificar una variable global desde dentro de una función. Para hacerlo, necesitamos utilizar la palabra clave global.

```
variable_global = "Soy global"

def modificar_global():
    global variable_global
    variable_global = "Modificado dentro de la función"
    print(variable_global)
modificar_global()  # Resultado: Modificado dentro de la función
print(variable_global)  # Ahora la variable global ha sido
modificada
```

Al utilizar global **variable_global** dentro de la función modificar_global, le estamos indicando a Python que queremos modificar la variable global **variable_global**.

10 Tipos y variables

Alcance de variables en funciones anidadas

Si definimos una función dentro de otra función, la función interna puede acceder a las variables locales de la función externa, **pero no puede modificarlas directamente**.

Sin embargo, si utilizamos la palabra clave **nonlocal**, podemos modificar variables de la función externa.

```
def funcion_externa():
    variable_externa = "Soy de la función externa"

    def funcion_interna():
        nonlocal variable_externa
        variable_externa = "Modificado por la función interna"
        print("Desde función interna:", variable_externa)

    funcion_interna()
    print("Desde función externa:", variable_externa)

funcion_externa()
```

En este ejemplo, **funcion_interna** puede acceder y modificar la variable **variable_externa** de **funcion_externa** debido al uso de **nonlocal**.

10 Tipos y variables

Tipo valor y referencia

En Python, los tipos de datos pueden ser divididos en dos categorías principales: tipos de valor y tipos de referencia

Esto es algo común en muchos lenguajes de programación. Entender las diferencias entre ambas es muy importante.

Tipos valor

Los tipos valor en Python son aquellos cuyas variables almacenan directamente el valor real. Esto significa que cuando asignamos una variable a otro valor, se crea una nueva copia de ese valor en la memoria.

Los tipos de valor más comunes en Python son los tipos básicos como enteros, flotantes, cadenas y booleanos.

Por ejemplo,

```
# Asignación de un entero a una variable
x = 10

# Se crea una nueva copia del valor 10 en la memoria para la
variable x
y = x

# Ahora x e y tienen valores independientes en la memoria
```

Cuando modificamos x o y, cada variable tiene su propio espacio en la memoria para almacenar su valor.

Tipos referencia

10 Tipos y variables

Los tipos referencia son aquellos cuyas variables almacenan una referencia (normalmente la dirección de memoria) al objeto real en lugar del valor real en sí mismo.

Esto significa que cuando asignamos una variable a otra, ambas variables apuntan al mismo objeto en la memoria.

Los tipos de referencia comunes incluyen listas, diccionarios, conjuntos y objetos personalizados.

Vamos a verlo con un ejemplo.

```
# Asignación de una lista a una variable
lista1 = [1, 2, 3]

# lista2 apunta al mismo objeto en la memoria que lista1
lista2 = lista1

# Modificar lista2 también modifica lista1
lista2.append(4)
print("lista1:", lista1)  # Resultado: [1, 2, 3, 4]
```

En este ejemplo, cuando modificamos **lista2**, también se modifica **lista1** porque ambas variables apuntan al mismo objeto en la memoria.

Pasando argumentos a funciones

Cuando pasamos variables a funciones, debemos tener en cuenta si son tipos de valor o tipos de referencia.

Pasando tipos de valor

Los tipos valor no se modifican cuando son pasados como parámetros, porque la función recibe una copia.

10 Tipos y variables

```
def duplicar_numero(num):
    num *= 2
    print("Dentro de la función:", num)

x = 10
duplicar_numero(x)
print("Fuera de la función:", x)  # x sigue siendo 10, no se
modificó
```

El valor de x no se modifica fuera de la función **duplicar_numero** porque es un tipo de valor.

Pasando tipos referencia

Los tipos referencia sí se modifican cuando son pasados como parámetros, porque la función recibe una copia de la referencia, que apunta a los mismos datos.

En este caso, **mi_lista** se modifica fuera de la función porque es un tipo de referencia.